MTD215

Timed Online Assignment – July Semester 2020

Application of C++ in Multimedia

Tuesday, 17 November 2020 1:00 pm – 3:30 pm

Time allowed: 2.5 hours

INSTRUCTIONS TO STUDENTS:

1. This Timed Online Assignment (TOA) contains **FOUR (4)** questions and comprises **SEVEN (7)** pages (including cover page).

2. You must answer **ALL** questions.

3. If you have any queries about a question, or believe there is an error in the question, briefly explain your understanding and assumptions about that question before attempting it.

4. You MUST **submit your answers via Canvas (similar to TMA submission) at the end time of this TOA** (as stated on this cover page). The 15 minutes grace period as shown on Canvas is strictly meant for technical issues encountered during submission. Thereafter, you will not be able to submit your answers and you will be considered as having withdrawn from the course. **No appeal will be allowed.**

5. You are to include the following particulars in your submission: Course Code, SUSS PI No., and Your Full Name. Name your submission file as TOACourseCode_FullName_StudentPI.

6. Your submission should consist of only one file and **must not exceed 500MB** in size. The file must be a Microsoft Word file saved in .doc or .docx format. All answers are to be typed. Flowcharts and graphs may be scanned or photographed and embedded in the Word file provided it does not exceed the file size limit of 500MB. Images of handwritten answers will not be marked.

7. To prevent plagiarism and collusion, your submission will be reviewed by Turnitin. The Turnitin report will only be made available to the marker and you will not be able to view it.

8. The University takes plagiarism and collusion seriously, and your Turnitin report will be examined thoroughly as part of the marking process.

Answer all questions. (Total 100 marks)

Question 1

Implement a C++ program that helps to explain the syntax and use of data structures such as structures and class.

(a) Define a structure named `harddisk` using the C++ language to hold

(i) A `string` representing the model of the hard disk.

(ii) A `char` representing the colour of the hard disk

(iii) A `float` representing the price of the hard disk.

(iv) An `integer` representing the capacity of the hard disk.

(5 marks)

(b) Implement a `main` function to demonstrate the use of the structure defined in part (a). The function declares an array of ten `harddisk` items. Define the last element in the array as follows:

model : "MTD215 Smart Drive",
colur : 'R'
pricec : $98.76, and
capacity : 2TB.

(8 marks)

(c) Modify the program in part (a) and (b) by converting the structure in part (a) to a C++ class. Implement a `main` function that performs the same tasks in part (b) using the C++ class you have just defined. Write your C++ code without changing the variable names described above. However, you can add your own variables if necessary but do not include other functions that are not needed here.

(12 marks)

Question 2

Identify suitable coding modules in a simple program that implement polymorphism. In this question you are asked to apply operator overloading which is a form of polymorphism.

An instance of **`ComplexNumber`** can be represented by `real` and `img`. Both `real` and `img` are integers.

Two instances of **`ComplexNumber`** can be added using the + operator. Consider the following example:

```
cn1 = 5 + 8i     i.e. real = 5 and img = 8
cn2 = 2 + 7i     i.e. real = 2 and img = 7
cn3 = cn1 + cn2 = 7 + 15i
```

Two instances of ComplexNumber can be multiplied using the * operator. The rule of multiplication is as follows:

```
(a + bi) * (c + di) = (ac – bd) + (ad + bc)i
```

Consider the following example:

```
cn1 = 5 + 8i     i.e. real = 5 and img = 8
cn2 = 2 + 7i     i.e. real = 2 and img = 7
cn4 = cn1 * cn2 = (10 - 56) + (35 + 16)i
                = -46 + 51i
```

Implement a C++ class to model **`ComplexNumber`** as specified below:

(a) Declare the data members `real` and `img`.

(2 marks)

(b) Write the constructor and destructor.

(5 marks)

(c) Overload the + operator.

(4 marks)

(d) Overload the * operator.

(4 marks)

(e) Overload the << operator so that the statement

```
cout << "cn1 = " << cn1;
```

will produce the result `cn1 = 5 + 8i`.

(5 marks)

(f) Write the `main()` function to construct some **`ComplexNumber`** instances and demonstrate the use of the `+`, `*` and << operators.

(5 marks)

Question 3

Discuss the concept of pointers, arrays and dynamically allocated memory by answering the following questions.

(a) Assume the following definition and initialisation:

```
int mtdArray[7] = {6, 8, 0, 1, 7, 5, 3};
```

What output does each of the following statements produce? Explain your answer.

(i) `cout << mtdArray[3] << endl;`

(ii) `cout << mtdArray[2] + mtdArray[4] << endl;`

(iii) `cout << mtdArray[mtdArray[4] - mtdArray[3]] << endl;`

(7 marks)

(b) Write C++ statements to carry out the following tasks:

- Declare a variable `c` and assign it the character `'M'`;
- Declare a variable `pChar` as a `char` pointer;
- Make `pChar` point to the location where `c` is;
- Increment `c` by 1 and store the result in the location where `pChar` is pointing.

What is the ouput when the statement below is executed?

```
cout << *pChar << endl;
```

(6 marks)

(c) The class `MyBook` has two data members:

- An integer, `numOfBooks`, represents the number of books.

- A dynamic location large enough to store the titles of a collection of books. The size of the collection is determined by `numOfBooks`. The location is referenced by the pointer `string* titles`.

(i) Define `MyBook` class with the appropriate data members, constructor and destructor.

(9 marks)

(ii) Write a `main` function to demonstrate how a `Mybook` instance can be created.

(3 marks)

Question 4

(a) Appraise the use of SDL for geometry rendering. SDL has primitive functions for drawing point, lines and rectangles. Consider the following `drawShape` function which is responsible for producing the output in Figure Q4(a).

```
void drawShape() {
    SDL_Rect fillRect = { 10, 10, 80, 80 };
    SDL_RenderFillRect(gRenderer, &fillRect);

    SDL_Rect outlineRect = { 110, 10, 80, 80 };
    SDL_RenderDrawRect(gRenderer, &outlineRect);

    SDL_SetRenderDrawColor(gRenderer, 0x00, 0x00, 0x00, 0xFF);
    SDL_RenderDrawLine(gRenderer, 250, 10, 210, 90);
    SDL_RenderDrawLine(gRenderer, 250, 10, 290, 90);
    SDL_RenderDrawLine(gRenderer, 210, 90, 290, 90);
}
```

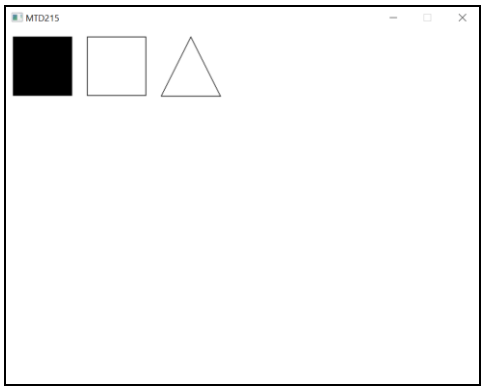

Figure Q4(a)

Re-write the `drawShape` function to produce each of following output:

(i) Sixteen Squares

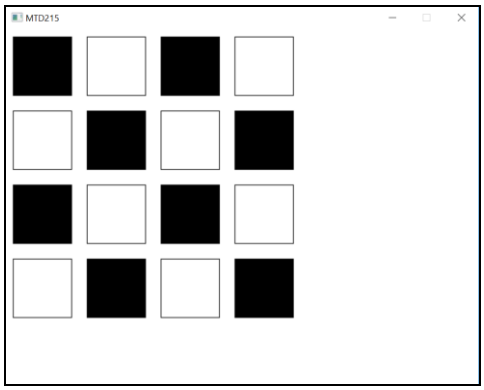

Figure Q4(a)(i)

(10 marks)

(ii) Ten Triangles

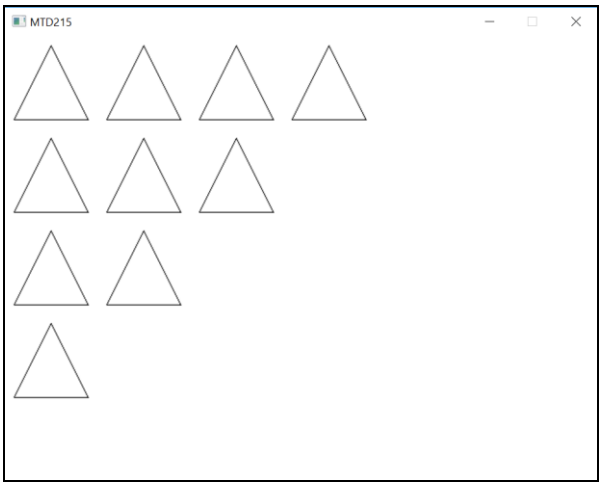

Figure Q4(a)(ii)

(8 marks)

(b) The `SDL_ttf` library provides a number of functions that allow a C++ program to initialise, load, open and close a true type font and render text onto a `SDL_Surface` object.

Identify and state the functions in the `SDL_ttf` library that are responsible for the above tasks.

(7 marks)

---- END OF PAPER ----